

Dynamic Task Offloading and Resource Allocation for Air-Ground Integrated Networks Based on MADDPG

Jianbin Xue, Peipei Mao[✉], Luyao Wang, Qingda Yu, Changwang Fan
(School of Computer and Communication, Lanzhou University of Technology,
Lanzhou 730050, China)

Abstract: With the rapid growth of connected devices, traditional edge-cloud systems are under overload pressure. Using mobile edge computing (MEC) to assist unmanned aerial vehicles (UAVs) as low altitude platform stations (LAPS) for communication and computation to build air-ground integrated networks (AGINs) offers a promising solution for seamless network coverage of remote internet of things (IoT) devices in the future. To address the performance demands of future mobile devices (MDs), we proposed an MEC-assisted AGIN system. The goal is to minimize the long-term computational overhead of MDs by jointly optimizing transmission power, flight trajectories, resource allocation, and offloading ratios, while utilizing non-orthogonal multiple access (NOMA) to improve device connectivity of large-scale MDs and spectral efficiency. We first designed an adaptive clustering scheme based on K-Means to cluster MDs and established communication links, improving efficiency and load balancing. Then, considering system dynamics, we introduced a partial computation offloading algorithm based on multi-agent deep deterministic policy gradient (MADDPG), modeling the multi-UAV computation offloading problem as a Markov decision process (MDP). This algorithm optimizes resource allocation through centralized training and distributed execution, reducing computational overhead. Simulation results show that the proposed algorithm not only converges stably but also outperforms other benchmark algorithms in handling complex scenarios with multiple devices.

Keywords: air-ground integrated network (AGIN); resource allocation; dynamic task offloading; multi-agent deep deterministic policy gradient (MADDPG); non-orthogonal multiple access (NOMA)

1 Introduction

With the rapid development of compute-intensive and latency-sensitive applications, efficiently processing massive amounts of data while ensuring low latency has become a critical challenge in wireless communication networks. In the context

of 5th-generation mobile communication technology (5G) networks, high-demand applications such as virtual reality, augmented reality, and autonomous driving require ultra-low latency, high reliability, and high bandwidth. However, due to limitations in device battery life and computing power, traditional network architectures still face significant challenges in handling these tasks [1, 2].

To address this issue, mobile edge computing (MEC)—a technology that deploys computing resources at the network edge—can effectively alleviate network congestion, reduce power

Manuscript received Dec. 20, 2024; revised Mar. 4, 2025; accepted Apr. 23, 2025. The associate editor coordinating the review of this manuscript was Dr. Han Hu. This work was supported by the Gansu Province Key Research and Development Plan (No. 23YFGA0062), and Gansu Provincial Innovation Fund (No. 2022A-215).

✉ Corresponding author. Email: maolucsy@163.com

DOI: [10.15918/j.jbit1004-0579.2024.124](https://doi.org/10.15918/j.jbit1004-0579.2024.124)

consumption, and lower latency, thereby supporting these compute-intensive and latency-sensitive applications [3, 4]. Nevertheless, existing fixed communication infrastructures face stability issues in special scenarios such as mountainous areas, deserts, remote regions, and large gatherings during festivals. Network coverage is often scarce in these areas, and it is easily affected by natural disasters, making reliable connectivity difficult to achieve.

Thus, how to quickly and flexibly deploy edge computing resources—especially by utilizing air-ground integrated networks (AGINs) to enhance system flexibility—has become a critical issue. Unmanned aerial vehicles (UAVs) have emerged as the core of AGINs due to their high flexibility, ease of deployment, and line-of-sight propagation characteristics [5]. UAVs can adjust their positions and trajectories according to demand to optimize network performance [6, 7] and can also act as aerial base stations in high-density user areas, such as busy roads or stadiums, thereby alleviating the burden on ground base stations and supporting users who cannot receive satisfactory service [8]. As a result, MEC and AGINs have emerged as key solutions to address these issues. Recently, the integration of MEC with UAVs in internet of things (IoT) and wireless communication networks has become an emerging research area.

In MEC-assisted integrated air-ground networks, as the number of edge users and task demands increases, spectrum resources may become insufficient. Traditional orthogonal multiple access (OMA) techniques can no longer meet the demands of large-scale access. Non-orthogonal multiple access (NOMA) technology uses superposition coding to allow different users to share the same spectrum, time, and spatial resources, and employs successive interference cancellation (SIC) to effectively reduce interference. This significantly improves spectrum utilization, supports large-scale access, alleviates traffic congestion, and reduces latency [9–11].

NOMA technology ensures efficient utilization of spectrum resources and is particularly suitable for addressing large-scale user access issues in edge computing and UAV networks [12, 13]. Therefore, integrating MEC with NOMA provides a more efficient solution for future wireless communication systems.

However, in optimizing UAV computation offloading and resource allocation, traditional optimization methods (such as convex optimization) often suffer from high computational complexity and insufficient precision due to the dynamic nature of the system environment and the complexity of multiple variables [14]. To address this challenge, intelligent optimization algorithms such as deep reinforcement learning (DRL) and reinforcement learning have been introduced. These methods model task offloading and resource allocation as a Markov decision process (MDP) to adapt to complex and dynamic communication environments. DRL-based approaches can continuously optimize offloading and resource allocation through ongoing learning and interaction with the environment, thereby enhancing system efficiency and stability.

To address the above challenges, this paper proposes an innovative MEC model supporting NOMA in an AGIN. In this model, ground mobile devices (MDs) offload tasks to UAVs equipped with MEC servers via NOMA, effectively enhancing spectrum utilization, reducing interference, and improving task offloading efficiency and reliability. Moreover, to mitigate unstable channel conditions caused by the bidirectional mobility of UAVs and mobile devices, an adaptive clustering algorithm based on K-Means is designed to further optimize resource allocation and reduce interference through NOMA. To cope with dynamic environments and the complexity of task offloading, a multi-agent deep reinforcement learning (MADDPG)-based algorithm is also proposed to optimize task offloading, UAV trajectory planning, and resource allocation. MADDPG facilitates cooper-

ation in a multi-agent environment by dynamically adjusting strategies to adapt to environmental changes, thereby effectively reducing the long-term computational cost for MDs and enhancing overall system performance.

1) We proposed an integrated air-ground MEC model supporting NOMA, where multiple UAVs equipped with MEC servers serve MDs. With NOMA, multiple users can share the same spectrum resources and utilize SIC to reduce interference, effectively enhancing spectrum utilization and access efficiency. The optimization objective is to jointly optimize UAV trajectories and MD task offloading strategies based on user locations and task information, thereby reducing each MD's latency and energy consumption and improving overall system performance.

2) Since the objective function involves multiple variables and is modeled as a mixed-integer nonlinear programming (MINLP) problem with nonconvex characteristics, we first designed an adaptive clustering algorithm based on K-Means to cluster devices for rational resource allocation. Subsequently, an MADDPG-based partial offloading algorithm is adopted to optimize task offloading and resource allocation. In particular, during UAV trajectory planning and offloading decision-making, MADDPG dynamically adjusts strategies, significantly enhancing system efficiency, optimizing the computational overhead of mobile devices, and ensuring the stability of the task offloading process.

3) Experimental results show that compared to other algorithms, the proposed scheme outperforms in several aspects, including multiple access schemes, UAV computing capability, system bandwidth, and MD offloading rates. Specifically, the scheme significantly improves intra-cluster resource utilization and effectively reduces latency, energy consumption, and computational overhead during task offloading. By combining NOMA and MADDPG, the system exhibits greater flexibility and efficiency in dynamic environments.

The remainder of this paper is organized as follows. In Section 2, we provide an overview of the related works. Section 3 presents the system model and problem formulation. Section 4 details the proposed solution algorithm. Section 5 provides simulation results and analysis. Section 6 concludes the paper.

2 Related Work

1) UAV-Based Computation Offloading

To address the issue of insufficient computational capability in user devices, early MEC research mainly focused on improving the performance of systems where computing services are provided by fixed base stations [15–17]. In [15], the authors proposed a method to alleviate overloaded servers by offloading the load to nearby idle servers. This method optimizes the control of migration/handover decisions, aiming to maximize the offloading rate for all users while minimizing the costs during the migration process. In [16], the authors further considered combining MEC offloading with device-to-device (D2D) offloading, proposing a learning-based task coordination offloading algorithm that aims to minimize system costs, including task delay and migration costs. This algorithm allows IoT devices to observe and evaluate the system costs of candidate edge nodes, thereby selecting the optimal edge node without needing complete offloading information. However, these studies only explored full offloading methods and did not fully consider the advantages of partial offloading and the potential of task allocation strategies to improve computational efficiency, and reduce energy consumption, or latency. To address this, in [17], the authors proposed a solution for optimizing offloading decisions, which improves long-term network utility through joint channel allocation and resource management in the context of random task scheduling and dynamic resource changes. However, the aforementioned studies did not account for the severe signal attenuation that may occur in ground communication due to

shadow fading and multipath effects in practical applications.

In recent years, research on UAV-assisted MEC in AGIN has attracted significant attention from scholars. In [18], the authors studied the cellular offloading problem assisted by a single UAV and proposed a spectrum reuse model, analyzing the impact of UAV altitude, transmission power, and offloading ratio on user downlink transmission rate. Since a single UAV has limited resources, cooperation among multiple UAVs is often necessary, making research on multi-UAV-assisted MEC more challenging. In [19], the authors investigated the subtask scheduling problem in a fault-tolerant UAV-assisted MEC emergency network and proposed the H-HSS algorithm to address topology reconstruction and subtask scheduling, minimizing the average completion time. In [20], the authors studied the smart offloading problem in multi-UAV-assisted MEC systems, modeling the offloading process as a Stackelberg game to optimize offloading and pricing strategies, while considering UAV energy consumption and user offloading willingness. In these studies, UAVs were used as temporary fixed base stations and did not consider that UAVs could change their positions based on real-time offloading strategies of users. In [21], the authors studied the three dimensional (3D) trajectory optimization problem of UAVs crossing multiple target locations, using Dubins curves to initialize smooth trajectories, aiming to maximize the computational efficiency of UAV-assisted MEC systems. In [22], the authors proposed a block coordinate descent (BCD)-based three-layer alternating iterative optimization method to obtain the optimal UAV trajectory, thereby reducing the cost function based on energy and delay. However, these studies did not consider optimizing UAV deployment locations according to the real-time motion trajectories and demands of MDs, nor did they explore how to effectively reduce the weighted

sum of task execution latency and energy consumption in such a dual-dynamic environment.

2) Application of NOMA in MEC

NOMA technology, as an effective method to enhance spectrum efficiency in MEC systems, has been widely studied. In [23], the authors proposed an NOMA-based UAV-MEC system security communication scheme targeting aerial eavesdroppers. By using successive convex approximation and block coordinate descent methods, they optimized the UAV trajectory to leverage the advantages of NOMA transmission. In [24], the authors investigated an MEC system assisted by multiple access points and UAVs, where joint optimization of task allocation, power allocation, and UAV trajectory minimized total energy consumption. In [25], the authors examined data security issues in UAV-assisted MEC under partial offloading mode for both time division multiple access (TDMA) and NOMA schemes, demonstrating that the partial offloading mode achieves greater secure computing capability compared to other offloading methods. NOMA technology significantly enhances spectrum utilization by allowing multiple users to share the same spectrum resources, making it especially suitable for bandwidth-limited MEC scenarios. However, despite its great potential in MEC, NOMA still faces challenges. For instance, dynamically allocating bandwidth in complex multi-user environments remains a key issue. As the number of users increases, it becomes critical to allocate bandwidth resources reasonably while ensuring fairness and quality, in order to avoid network performance degradation due to spectrum overlap and interference.

3) Application of DRL in AGINs

With the rapid development of DRL technology, an increasing number of studies are applying it to optimize task offloading and resource management in UAV-assisted MEC networks. For example, in [26], the authors proposed a deep Q-network (DQN) algorithm with virtual agents to solve the multi-hop offloading problem in

UAV-assisted MEC networks in dynamic and uncertain environments. Additionally, some studies use DRL to dynamically adjust UAV trajectories to further improve network performance. In [27], the authors proposed a DRL-based technique that optimizes UAV flight trajectories and data collection performance using deep neural networks to improve system throughput. This approach replaces the traditional Q-value function of DQL algorithms with DNN to model the relationship between action and state spaces, thereby improving convergence rate and stability. In [28], the authors transformed the UAV tracking control task into an MDP and designed a deep deterministic policy gradient (DDPG)-based control algorithm to solve the continuous action MDP problem. While [27] and [28] mainly focus on single UAV scenarios, [29] proposed a UAV-assisted MEC network framework that independently manages the trajectories of each UAV, jointly optimizing the geographical fairness of ground user devices, the load fairness of each UAV, and the overall energy consumption of user devices. However, existing DRL-based research typically adopts centralized architectures, which not only increase communication overhead and energy consumption but also pose significant privacy risks. Therefore, this paper proposes an MADDPG algorithm, designed to improve multi-agent cooperation efficiency through centralized training and distributed execution, and to optimize resource allocation, thereby effectively addressing the limitations of centralized DRL architectures.

3 System Model and Problem Formulation

3.1 Network Model

As shown in Fig. 1, we considered an air-ground integrated MEC network model based on NOMA transmission. The model consists of multiple UAVs and several IoT mobile devices forming an uplink NOMA-UAV system. We assume that the system has M MDs and U UAVs, and define the

set of all mobile devices as $m \in M = \{1, 2, \dots, M\}$ and the set of all UAVs as $u \in U = \{1, 2, \dots, U\}$. Each UAV is equipped with a micro MEC server to facilitate the offloading and computation of MD tasks. To ensure good transmission links between UAVs and MDs and timely feedback for task offloading requests, MDs can be clustered based on their spatial positions and associated with specific UAVs. This enables UAVs to optimize computational resource utilization and better handle task offloading requests from MDs. It is assumed that all MDs are divided into the same number of mobile device clusters $C_u \in C = \{C_1, C_2, \dots, C_U\}$, $C = U$ as UAVs, and a UAV is assigned to each cluster, which ensures that each MD is covered and not served by multiple UAVs.

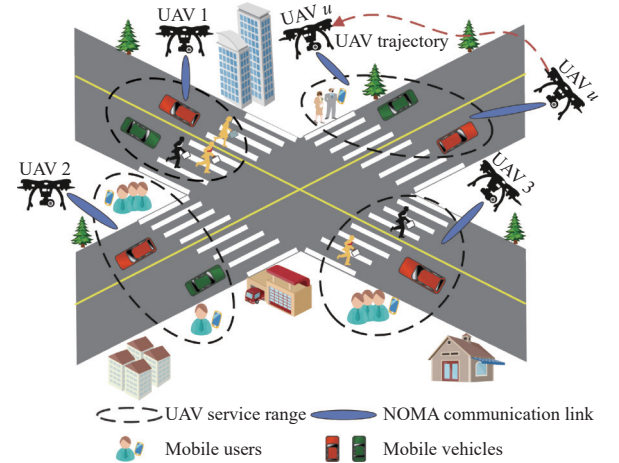


Fig. 1 NOMA-based air-ground integrated MEC network model

In this paper we also utilize TDMA technique to offload tasks generated by MDs to UAVs via the uplink transmission link. All UAVs complete a flight task within a time period T and finish the task computation offloading of MDs in each time slot. The entire service time period is discretized into time slots of length Δt , and the set of time slots is denoted as $t \in T = \{0, 1, 2, \dots, T\}$. Within each time slot Δt , the channel is assumed to remain static. We assume that UAVs and MDs move randomly in a three-dimensional dynamic space. In time slot t , the initial position of UAV u is denoted as $q_u^{\text{UAV}}(t) = \{X_u^{\text{UAV}}(t), Y_u^{\text{UAV}}(t), H_u^{\text{UAV}}\}$, and the position

of MD m is denoted as $q_m^{\text{MD}}(t) = \{X_m^{\text{MD}}(t), Y_m^{\text{MD}}(t), 0\}$. In the next time slot, the coordinates and tasks of the MDs may update randomly within a certain range, and MDs will be re-match with UAVs based on their positions and tasks. The UAV starts from the cluster's initial point and flies to the endpoint, completing the computation offloading of all MDs within the cluster and optimizing its own trajectory. Therefore, we assume that both UAVs and MDs move at a constant speed, with the maximum flight speed of UAVs and the maximum movement speed of MDs denoted as $V_{\text{UAV}}^{\text{max}}$ and $V_{\text{MD}}^{\text{max}}$, respectively. However, during the actual UAV service process, due to the random movement of MDs within the cluster, the distance between users in the cluster may increase over time, leading to a deterioration in the communication quality between UAVs and MDs. Thus, the distance threshold in MD m and MD k in the cluster is set as $d_{m,k}^{\text{max}}$. When the distance between two MDs exceeds this threshold, the cluster will be redefined to maintain a good

communication state between the MDs and UAV. The parameters used in this paper are shown in Tab. 1.

3.2 Task Model

We assume that all MDs generate a delay-sensitive and computation-intensive task within the time slot t , which is oriented towards data partitioning. Due to the bit independence, it can be arbitrarily partitioned into multiple subtasks with no mutual overlap among them, enabling in an easy way the parallel processing and distributed computation of the task with negligible partition cost [30]. Therefore, we can describe the task of MD m as a binary group

$$\Phi_m = \{D_m(t), \Omega_m(t)\} \quad (1)$$

where $D_m(t)$ denotes the size of the data generated by the m th MD within the time slot t , in bit; $\Omega_m(t)$ denotes the number of CPU cycles of the m th MD required to process 1 bit of data within the time slot t , in cycles/bit. We consider that if the task is computed in parallel, the local and UAV computational resources can be processed jointly at the same time, which in turn reduces the computational overhead of the MDs, and thus a partial computation offloading model is constructed for each MD. Here we introduce the offloading factor $\zeta_{m,u}(t) \in [0, 1]$, and then there are three ways to process the tasks for MD m : 1) all tasks are processed locally by MD m ; 2) partial tasks are processed by UAV u ; and 3) all tasks are processed by UAV u . Therefore, the size of the task computed by MD m when we compute the task by adopting partial offloading is denoted as

$$D_m(t) = \begin{cases} 0, & \text{full offloading} \\ [1 - \zeta_{m,u}(t)] D_m(t), & \text{partial offloading} \\ D_m(t), & \text{local computing} \end{cases} \quad (2)$$

Therefore, the computation offloading of tasks involves multiple influences like local execution, transmission link, UAV execution, etc., which will be discussed in detail in the following sections.

Tab. 1 System model symbols

Notation	Description
M/U	The total number of MDs/ UAVs
T	Offloading service duration
$q_u^{\text{UAV}}/q_m^{\text{MD}}$	Coordinates of UAV u / MD m
Φ_m	The task of MD m
D_m	The size of the task generated by MD m
Ω_m	The number of cycles required for 1 bit
$\zeta_{m,u}$	Offloading decision factor of MD m
$d_{u,m}/\theta_{u,m}$	The distance/azimuth between MD and UAV
$p_{m,u}^{\text{LoS}}/p_{m,u}^{\text{NLoS}}$	Occurrence probability of LoS/NLoS
$l_{m,u}^X$	Average pass loss between MD m and UAV u
$g_{m,u}$	Channel gain between MD m and UAV u
$\psi(t)$	Fading coefficient between MD m and UAV u
$a_{m,u}$	Serving indicator
$\sigma_{m,u}$	Additive white Gaussian noise
$G_{m,u}$	Equivalent channel gain
b_u	Bandwidth allocated to each cluster
f_m^{MD}	The computing capability available at each MD
$\varepsilon_m/\varepsilon_u$	Energy coefficient
$f_{m,u}^{\text{UAV}}$	Computation resources allocated to the m th MD
$\beta_{m,u}$	Computing resource allocation ratios
$V_{\text{UAV}}^{\text{max}}/V_{\text{MD}}^{\text{max}}$	Maximum speed of the UAV/ MD
δ_t/δ_e	Weighting factor for time/ energy consumption

3.3 Communication Model

1) A2G Propagation Model

In realistic scenarios, UAVs need to constantly adjust their flight attitude for better communication due to obstacle occlusion factors such as tall buildings and trees. Therefore, we consider two line-of-sight link states of line of sight (LoS) and non-line of sight (NLoS) to construct the air-to-ground (A2G) path loss model during the A2G communication between each UAV and the MDs in the cluster [31]. Where we only consider the task uplink and ignore the downlink, the distance $d_{u,m}(t)$ and elevation angle $\theta_{u,m}(t)$ between MD m and UAV u in the time slot t can be calculated as

$$\begin{cases} d_{m,u}(t) = \|q_m^{\text{MD}}(t) - q_u^{\text{UAV}}(t)\| \\ \theta_{u,m}(t) = \arcsin[H_u^{\text{UAV}}/d_{u,m}(t)] \end{cases} \quad (3)$$

where $\|\cdot\|$ is the Euclidean paradigm. Therefore, the probability of LoS connection between MD m and UAV u is

$$P_{m,u}^{\text{LoS}}(t) = \frac{1}{1 + \mu_a \exp\{\mu_b - [\theta_{m,u}(t) - \mu_a]\}} \quad (4)$$

where μ_a and μ_b are parameters that depend on the types of propagation environments (such as dense urban, suburban, rural, etc.). Similarly, we can obtain the probability of NLoS connection as $P_{u,m}^{\text{NLoS}}(t) = 1 - P_{m,u}^{\text{LoS}}(t)$. The average path loss equation between MD m and UAV u can be expressed as

$$l_{m,u}^{\chi}(t) = l_{m,u}^{\text{loss}}(t) + \mathfrak{S}_{\chi} \quad (5)$$

where χ is denoted as both LoS and NLoS propagation types. Therefore, the free space path loss between MD m and UAV u can be expressed as

$$l_{m,u}^{\text{loss}}(t) = 20 \log_2 d_{m,u}(t) + 20 \log_2 f_c + 20 \log_2 (4\pi/V_c) \quad (6)$$

where f_c denotes the system carrier frequency; V_c denotes the speed of light. Therefore, the path loss between MD m and UAV u is expressed as

$$\begin{aligned} l_{m,u}(t) &= P_{m,u}^{\text{LoS}}(t)l_{m,u}^{\text{LoS}}(t) + P_{u,m}^{\text{NLoS}}(t)l_{m,u}^{\text{NLoS}}(t) = \\ &P_{m,u}^{\text{LoS}}(t)l_{m,u}^{\text{loss}}(t) + P_{m,u}^{\text{LoS}}(t)\mathfrak{S}_{\text{LoS}} + \\ &P_{u,m}^{\text{NLoS}}(t)l_{m,u}^{\text{loss}}(t) + P_{u,m}^{\text{NLoS}}(t)\mathfrak{S}_{\text{NLoS}} = \\ &l_{m,u}^{\text{loss}}(t) + P_{m,u}^{\text{LoS}}(t)\mathfrak{S}_{\text{LoS}} + P_{u,m}^{\text{NLoS}}(t)\mathfrak{S}_{\text{NLoS}} \end{aligned} \quad (7)$$

where $\mathfrak{S}_{\text{LoS}}$ and $\mathfrak{S}_{\text{NLoS}}$ represent the excessive path loss for LoS and NLoS links, respectively. At the t moment, considering the effect of channel fading, the channel gain between the MD m and UAV u can be expressed as

$$g_{m,u}(t) = \psi(t)10^{-l_{m,u}(t)/10} \quad (8)$$

where $\psi(t)$ denotes the fading coefficient between MD m and UAV u [32].

2) Channel Model

At the t moment, the communication link relationship between UAVs and all MDs in the system can be directly represented by a matrix as

$$\mathbf{A} = \begin{bmatrix} a_{1,1}(t) & a_{1,2}(t) & \cdots & a_{1,U}(t) \\ a_{2,1}(t) & a_{2,2}(t) & \cdots & a_{2,U}(t) \\ \vdots & \vdots & \vdots & \vdots \\ a_{M,1}(t) & a_{M,2}(t) & \cdots & a_{M,U}(t) \end{bmatrix} \quad (9)$$

We regard $\mathbf{A}$ as the flag where UAVs provide services for MDs, then when $a_{m,u}(t) = 1$ indicates that the UAV u provides computation offloading services for MD m , otherwise $a_{m,u}(t) = 0$. Therefore, the superimposed transmit signals from all MDs in the cluster that can be received by the UAV in each cluster at the time slot t as

$$\tilde{\mathbf{y}}_u = \sum_{m=1}^M a_{m,u}(t) \sqrt{P_{m,u}(t)} \tilde{\mathbf{y}}_{m,u}(t) \quad (10)$$

where $\tilde{\mathbf{y}}_{m,u}(t)$ denotes the transmitted signal from the MD m to the UAV u , and $P_{m,u}(t)$ is denoted as the transmitted power of the MD m . Therefore, the superimposed signal that can be received at the UAV u is denoted as

$$\ddot{\mathbf{y}}_{m,u}(t) = g_{m,u}(t) \tilde{\mathbf{y}}_{m,u}(t) + I_{m,u}^{\text{inter}}(t) + I_{m,u}^{\text{intra}}(t) + \sigma_{m,u}(t) \quad (11)$$

where $g_{m,u}(t) \tilde{\mathbf{y}}_{m,u}(t)$ is the desired signal; $I_{m,u}^{\text{inter}}(t)$ denotes the inter-cluster interference from UAVs in clusters other than UAV u to MD m when MD m in a cluster communicates with UAV u . $I_{m,u}^{\text{intra}}(t)$ denotes the intra-cluster interference from other MDs in the cluster except the MD m to the UAV u when MD m in the cluster communicates with the UAV u ; $\sigma_{m,u}(t)$ is Gaussian white

noise. Therefore, the inter-cluster interference from other UAVs to the MD m is given as follows

$$I_{m,u}^{\text{inter}}(t) = \sum_{u=1, j \neq u}^U g_{m,j}(t) \sqrt{P_{m,j}(t)} \tilde{y}_j = \sum_{u=1, j \neq u}^U \sum_{m=1}^M a_{m,j}(t) g_{m,j}(t) P_{m,j}(t) \tilde{y}_{m,j}(t) \quad (12)$$

where $g_{m,j}(t)$ is defined as the channel gain between the MD m and the UAV j ; $P_{m,j}(t)$ is defined as the transmit power from the MD m to the UAV j ; $a_{m,j}(t)$ and $\tilde{y}_{m,j}(t)$ denote whether the UAV j provides computation offloading service to the MD m and the MD m transmit signal to the UAV j , respectively.

In this paper, it is considered that SIC technique will be utilized in the NOMA-based task computation offloading system. It involves sequentially adjudicating devices in the received signal, performing amplitude recovery for each device, and subtracting the multiple access interference (MAI) caused by the signals of the device from the received signal. This process continues

$$\text{SINR}_{m,u}(t) = \frac{a_{m,u}(t) g_{m,u}(t) \sqrt{P_{m,u}(t)}}{I_{m,u}^{\text{inter}}(t) + I_{m,u}^{\text{intra}}(t) + \sigma_{m,u}(t)} = \frac{a_{m,u}(t) g_{m,u}(t) \sqrt{P_{m,u}(t)}}{\sum_{u=1, j \neq u}^U g_{m,j}(t) \sqrt{P_{m,j}(t)} + \sum_{k=m+1}^M a_{k,u}(t) g_{k,u}(t) \sqrt{P_{k,u}(t)} + \sigma_{m,u}(t)} \quad (15)$$

Then the transmission rate between MD m and UAV u can be calculated as

$$R_{m,u}(t) = b_u \log_2(1 + [\text{SINR}_{m,u}(t)]^2) \quad (16)$$

$$b_u = \frac{B^{\text{total}}}{C_U} \quad (17)$$

where b_u denotes the sub-bandwidth allocated to each cluster based on the number of clusters for the total system bandwidth, and all MDs within each cluster share the same spectrum resource.

3.4 Partial Offloading Model

In general, the offloading strategy adopted by a mobile device in a multi-UAV assisted MDs' task computation offloading system will directly affect its computational overhead. Through Eq. (2) we

iteratively until all MAIs are eliminated. Since the positions of UAV and MDs in each cluster are time-variable, we should determine the decoding order based on the channel gains and the intra-cluster interference received by the UAV, and then the equivalent channel gains are sorted from the largest to the smallest which can be expressed as

$$g_{1,u}(t) \geq g_{2,u}(t) \geq \dots \geq g_{M,u}(t) \quad (13)$$

According to the uplink NOMA decoding rules, once the UAV u receives uplink signals transmitted by all the mobile devices in the cluster, it will first decode the signals with better channel gains and regard the other signals with poorer channel gains as interference. Therefore, the intra-cluster interference caused by other MDs in the cluster to the MD m is given by the following equation

$$I_{m,u}^{\text{intra}}(t) = \sum_{k=m+1}^M a_{k,u}(t) g_{k,u}(t) \sqrt{P_{k,u}(t)} \tilde{y}_{k,u}(t) \quad (14)$$

The signal-to-interference noise ratio (SINR) of the signal in the cluster where the UAV u decodes the MD m is

can see that the MD's task computation model is determined by three different offloading strategies.

1) Local Computing Model

Assume that the size of the MD m 's computational capability during local computation is f_m^{MD} in cycles/s, which is related to the MD m 's chip architecture. Therefore, we can compute the average delay and average energy consumption incurred by the MD m choosing to compute partial tasks locally at the time slot t , respectively

$$T_m^{\text{loc}}(t) = [1 - \zeta_{m,u}(t)] \frac{D_m(t) \Omega_m(t)}{f_m^{\text{MD}}(t)} \quad (18)$$

$$E_m^{\text{loc}}(t) = \varepsilon_m [f_m^{\text{MD}}(t)]^3 T_m^{\text{loc}}(t) = \varepsilon_m [1 - \zeta_{m,u}(t)] [f_m^{\text{MD}}(t)]^2 D_m(t) \Omega_m(t) \quad (19)$$

where ε_m is an effective switching capacitor depending on the MD m 's CPU architecture.

2) UAV Computational Model

When MD m chooses to offload partial tasks to UAV u at the time slot t , the average delay and average energy consumption incurred by task transmission between UAV u and MD m are expressed, respectively

$$T_{m,u}^{\text{tran}}(t) = \frac{\zeta_{m,u}(t)D_m(t)}{R_{m,u}(t)} \quad (20)$$

$$E_{m,u}^{\text{tran}}(t) = P_{m,u}(t)T_{m,u}^{\text{tran}}(t) = \frac{P_{m,u}(t)\zeta_{m,u}(t)D_m(t)}{R_{m,u}(t)} \quad (21)$$

This gives us the average delay and the average energy consumption incurred by the UAV u in the computational task denoted respectively

$$T_{m,u}^{\text{com}}(t) = \frac{\zeta_{m,u}(t)D_m(t)\Omega_m(t)}{f_{m,u}^{\text{UAV}}(t)} \quad (22)$$

$$E_{m,u}^{\text{com}}(t) = \varepsilon_u [f_{m,u}^{\text{UAV}}(t)]^3 T_{m,u}^{\text{com}}(t) = \varepsilon_u \zeta_{m,u}(t) [f_{m,u}^{\text{UAV}}(t)]^2 D_m(t) \Omega_m(t) \quad (23)$$

$$f_{m,u}^{\text{UAV}}(t) = \beta_{m,u} F_{\text{UAV}}^{\text{max}} \quad (24)$$

where $f_{m,u}^{\text{UAV}}$ denotes the computational resources allocated by the UAV u based on the number of MDs in the cluster and the volume of offloading tasks of the MD m , $\beta_{m,u}$ is the ratio of the allocated computational resources; and ε_u denotes the effective switching capacitance based on the CPU architecture of the UAV u on its own.

Since the output results of various computation-intensive tasks are much smaller than the inputs, the delay and energy consumption spent on downlink transmission can be ignored.

3.5 UAV Energy Consumption Model

Since the UAV has limited energy and has to accomplish the task requests of all the mobile devices in the cluster within the period T , it is necessary to obtain the initial position $q_{0,u}^{\text{UAV}} = \{X_{0,u}^{\text{UAV}}, Y_{0,u}^{\text{UAV}}, H_{0,u}^{\text{UAV}}\}^T$ and the final position $q_{F,u}^{\text{UAV}} = \{X_{F,u}^{\text{UAV}}, Y_{F,u}^{\text{UAV}}, H_{F,u}^{\text{UAV}}\}^T$ of the UAV u in the cluster. Hence, we can obtain

$$q_u^{\text{UAV}}(0) = \{X_u^{\text{UAV}}(0), Y_u^{\text{UAV}}(0), H_u^{\text{UAV}}\} = q_{0,u}^{\text{UAV}} \quad (25)$$

$$q_u^{\text{UAV}}(T) = \{X_u^{\text{UAV}}(T), Y_u^{\text{UAV}}(T), H_u^{\text{UAV}}\} = q_{F,u}^{\text{UAV}} \quad (26)$$

In each cluster, since each UAV may have a maximum flight distance due to the limited flight speed of the UAV, the maximum flight distance and speed of the UAV u should satisfy the following conditions

$$\|q_u^{\text{UAV}}(t+1) - q_u^{\text{UAV}}(t)\| \leq V_{\text{UAV}}^{\text{max}} \Delta t \quad (27)$$

$$V_{\text{UAV}}(t) = \frac{\|q_u^{\text{UAV}}(t+1) - q_u^{\text{UAV}}(t)\|}{\Delta t}, \forall t \quad (28)$$

The energy consumption generated by UAVs in the air mainly consists of the propulsion energy consumption during flight and the energy consumption associated with the establishment of communication links with the former dominating. Usually, the propulsion energy consumption is of four to five orders of magnitude larger than the communication energy consumption, which has been discussed in the previous section for UAV and will be discussed in this section for UAVs' flight energy consumption [33]. Therefore, the flight power consumption of a UAV can be expressed as

$$P_u^{\text{fly}}(\|V_{\text{UAV}}(t)\|) = P_u^{\text{blade}} \left(1 + \frac{3\|V_{\text{UAV}}(t)\|^2}{v_{\text{tip}}^2}\right) + \frac{1}{2}d_0\rho s A \|V_{\text{UAV}}(t)\|^3 + P_u^{\text{induced}} \left(\sqrt{1 + \frac{\|V_{\text{UAV}}(t)\|^4}{4V_{0,\text{UAV}}^4}} - \frac{\|V_{\text{UAV}}(t)\|^2}{2V_{0,\text{UAV}}^2}\right)^{\frac{1}{2}}, \forall t \quad (29)$$

where the specific definition of UAVs in flight is expressed as follows: P_u^{blade} denotes the blade type power in hover, P_u^{induced} denotes the induced power in hover, v_{tip} denotes the tip speed of the UAV rotor blade, $V_{0,\text{UAV}}$ denotes the average rotor induced speed in hover, d_0 denotes the fuselage drag ratio, ρ denotes the air density, s denotes the ruggedness of UAV rotor, and A denotes the area of UAV rotor disk. Then the average flight energy consumption of the UAV u in each time slot t is

$$E_u^{\text{fly}}(t) = \Delta t \frac{1}{T} \sum_{t=1}^T P_u^{\text{fly}}(\|V_{\text{UAV}}(t)\|) \quad (30)$$

To ensure the system energy fairness, the UAV flight energy consumption weighting is set to ϑ [34]. Combining Eqs. (21) and (23) shows that the generated average energy consumption between UAV u and MD m within the time slot t is

$$E_{m,u}^{\text{total}}(t) = E_{m,u}^{\text{tran}}(t) + E_{m,u}^{\text{com}}(t) + \vartheta E_u^{\text{fly}}(t) \quad (31)$$

3.6 Problem Optimization

In this paper, we were investigating the problem of minimizing the average computational overhead incurred by each MD within a cluster in an uplinked NOMA-UAV system when it finishes the task computation offloading. Since the MDs adopt a partial offloading strategy, the MD's local computation can be simultaneously executed with the transmission task to the UAV as well as the UAV computation task. Combining Eqs. (20) and (22) gives the total delay incurred by the MD m in accomplishing the task computation

$$T_m^{\text{total}}(t) = \max\{T_m^{\text{loc}}(t), T_{m,u}^{\text{tran}}(t) + T_{m,u}^{\text{com}}(t)\} \quad (32)$$

Therefore, combining Eqs. (31) and (32), the weighting sum of the average delay and average energy consumption between UAV u and MD m can be obtained as $\prod_{m,u}^{\text{total}}(t)$, which is a unitless parameter reflecting the performance of delay and energy consumption tradeoffs of the offloading strategy. Then the average computational cost $\prod_{m,u}^{\text{total}}(t)$ of MD m in time slot t can be expressed as

$$\prod_{m,u}^{\text{total}}(t) = \frac{1}{TM} \sum_{m=1}^M \sum_{t=1}^T \delta_t T_m^{\text{total}}(t) + \delta_e E_{m,u}^{\text{total}}(t) \quad (33)$$

In order to maximize the resource utilization of the system while guaranteeing the quality of experience (QoE) of MD m , we introduced the delay weighting factor δ_t and the energy consumption weighting factor δ_e in (35), and their relationship can be expressed as

$\delta_t, \delta_e \in [0, 1]$, $\delta_t + \delta_e = 1$. If $\delta_t > \delta_e$, it indicates that the optimization problem is more biased towards optimizing the delay of the MD m than towards optimizing the energy consumption of the MD m , and vice versa. It further shows that these factors can be adjusted according to different delay and energy requirements to obtain the corresponding optimization.

Therefore, we can jointly optimize the problem of minimizing the average computational overhead of each MD in the cluster by jointly optimizing the transmission power $P = \{P_{m,u}(t), P_{k,u}(t), \forall m, k \in M, u \in U\}$ of the MD in the cluster, the flight trajectory $q = \{q_u^{\text{UAV}}(0), q_u^{\text{UAV}}(T), \forall u \in U\}$ of the UAV, the proportion of computational resources $\beta = \{\beta_{m,u}, \forall m \in M, u \in U\}$ allocated by the UAV to the MD in the cluster, and the offloading proportion $\zeta = \{\zeta_{m,u}(t), \forall m \in M, u \in U\}$ in each time slot t . Thus, the mathematical expression of the optimization problem can be expressed as

$$\begin{aligned} & \min_{\{P, q, \beta, \zeta\}} \prod_{m,u}^{\text{total}}(t) \\ & \text{s.t. C1: } q_u^{\text{UAV}}(0) = q_{0,u}^{\text{UAV}}, q_u^{\text{UAV}}(T) = q_{F,u}^{\text{UAV}}, \forall u \in U \\ & \text{C2: } \|q_u^{\text{UAV}}(t+1) - q_u^{\text{UAV}}(t)\| \leq V_{\text{UAV}}^{\text{max}} \Delta t, \forall u \in U \\ & \text{C3: } 0 \leq P_{m,u}(t) \leq P_{\text{MD}}^{\text{max}}, \forall m \in M, u \in U \\ & \text{C4: } 0 \leq P_{k,u}(t) \leq P_{\text{MD}}^{\text{max}}, \forall k \in M, u \in U \\ & \text{C5: } 0 \leq \beta_{m,u} \leq 1 \\ & \text{C6: } 0 \leq \sum_{m=1}^M \beta_{m,u} f_{m,u}^{\text{UAV}}(t) \leq F_{\text{UAV}}^{\text{max}}, \forall m \in M, u \in U \\ & \text{C7: } 0 \leq \sum_{u=1}^{C_U} b_u \leq B^{\text{total}}, 0 \leq b_c \leq B^{\text{total}}, \forall C_u \in C \\ & \text{C8: } 0 \leq \zeta_{m,u}(t) \leq 1, \forall m \in M, u \in U \\ & \text{C9: } a_{m,u}(t) = \{0, 1\}, \forall m \in M, u \in U \\ & \text{C10: } 0 \leq \delta_t \leq 1, 0 \leq \delta_e \leq 1 \end{aligned} \quad (34)$$

In optimization problem (34), constraints C1 and C2 impose movement limitations on the UAV u , ensuring that the movement distance does not exceed the maximum distance $V_{\text{UAV}}^{\text{max}} \Delta t$, which is determined by the UAV's maximum flight speed $V_{\text{UAV}}^{\text{max}}$. Constraints C3 and C4 restrict the transmission power of all MDs within each

cluster, ensuring it does not exceed the maximum transmission power $P_{MD}^{\max}$; constraints C5 and C6 are the constraints on the allocation of computational resources by the UAV in the cluster to all the MDs in charge of the cluster. The computational resources allocated by the UAV in the cluster for all the MDs which it is responsible for do not exceed its own maximum computational resource capacity $F_{UAV}^{\max}$; constraint C7 is the sub-bandwidth constraint assigned to each cluster and the cumulative sum of sub-bandwidths of all clusters does not exceed the total system bandwidth B^{total} ; constraint C8 is the adoption of a partial offloading strategy for each MD; constraint C9 represents the communication service flags of the binary between the UAV and the MD; and constraint C10 represents the constraints on the delay weighting factor and the energy consumption weighting factor.

Due to the nonconvex nature of the problem (34), it is usually considered an NP-Hard problem which is difficult to solve. Meanwhile, the UAV and MD in the system with a random movement nature cause the formulation problem to have high dynamics. Hence, traditional convex optimization algorithms face challenges and high computational complexity in solving the formulation problem. To address this, we employed an optimization algorithm based on multi-agent deep reinforcement learning that enables the agents to interact with the environment and learn from these interactions to find solutions.

4 Proposed Solution

This section introduces an effective approach for the task computation offloading model in a multi-UAV-assisted mobile device environment. The proposed scheme involves a two-step process. First, to enhance resource utilization and address communication service matching between UAVs and ground MDs, we developed an adaptive clustering algorithm based on K-Means. This algorithm clusters MDs based on their locations and

the capacity of computational resources of each UAV, aiming to reduce interference among clusters. Each cluster is then assigned with a UAV to establish a three-dimensional space communication link between MDs and UAVs. Next, we applied the MADDPG algorithm to handle the computational overhead resulting from the computation offloading of each MD within the cluster. That is, for each UAV, the DDPG algorithm is devised to address the joint optimization problem, which enables the UAV to track the movement of the MD in a real-time manner and reduces both delay and energy consumption of the computational offloading of each MD task, ultimately minimizing the goal of the average computational overhead for each MD.

4.1 Adaptive Cluster Algorithm

We assume that a UAV knows all the coordinate information of the ground MDs, defines the set of coordinates as $L = \{L_1, L_2, \dots, L_M\}$, and broadcasts the MD partition to all UAVs in the system through the control channel. To effectively manage the MDs in the system, we partition all MDs into clusters based on their spatial distances. To avoid interference, the clusters should be as dispersed as possible, ensuring that each cluster has a reasonable spatial coverage. Additionally, we considered the distance between UAVs and MDs, as well as the computational resource capacity of UAVs, which play a crucial role in the MD clustering process. Therefore, we employed an adaptive clustering algorithm based on the traditional K-Means method. This algorithm not only can effectively group MDs into C clusters but also ensures that the number of MDs in each cluster does not exceed the upper limit value to balance the computational load of the UAVs and to achieve a lower complexity. The core idea of the K-Means algorithm is to minimize the distance between the centroid and each MD within a cluster to optimize clustering performance. Thus, the cluster representation of the least sum of squared errors (SSE) is expressed as

$$\text{SSE} = \arg \min \sum_{m=1}^M \sum_{L \in C_u} \|L_m - \varphi_u\|^2 \quad (35)$$

where φ_u is the centroid of the cluster C_u and can be denoted as

$$\varphi_u = |C_u|^{-1} \sum_{L \in C_u} L \quad (36)$$

We can define the set of centroids of each cluster $\varphi_u \in \varphi = \{\varphi_1, \varphi_2, \dots, \varphi_U\}$ for each cluster. In this algorithm, the initial cluster centers are placed at randomly chosen positions, and the selection of cluster centers is optimized using the K-Means++ initialization method to reduce instability caused by random initialization. K-Means++ selects new cluster centers from MDs that are farther away from existing centers, ensuring a more uniform initial cluster distribution and avoiding local optima. Each MD is assigned to the nearest cluster based on its distance to the cluster center, while the distance between each MD and the current center is computed to determine the probability of that MD being selected as the next cluster center. This process is iterated until all cluster centers are determined and each cluster reaches the designated upper limit of MDs.

Considering the dynamic movement of UAVs and MDs, the clustering results will change continuously. Therefore, whenever the position of an MD or UAV changes, the algorithm dynamically adjusts the clustering. When the distance between MDs and UAVs exceeds a certain threshold, the clusters are re-partitioned to ensure system efficiency and stability.

Algorithm 1 Adaptive Algorithm Based on K-Means

Input: Coordinate information of MD L ; number of generated clusters C ; maximum number of MDs per cluster δ ; maximum number of iterations N ; centroid set of each cluster φ .

Output: C_U clusters are $\{C_1, C_2, \dots, C_U\}$.

- 1: Randomly select one sample L_m from L as the initial centroid $\varphi_1 = L_m$
- 2: Add sample L_m to the first cluster $C_1 = C_1 \cup \{L_m\}$

and remove from set L , update L

3: for $i = 1, 2, \dots, M - 1$ do

4: Calculate the sum of the distances between the remaining MDs and the current cluster centroid according to $t_{i,j} = \sum_{j=1}^i \|L_m - \varphi_j\|$

5: Select the MD j with the largest distance according to $j = \text{argmax}(t_{i,j})$

6: Take the sample L_j as the centroid $\varphi_{i+1} = L_j$ of the $i + 1$ cluster

7: Add the sample L_j to the first cluster $C_{i+1} = C_{i+1} \cup \{L_j\}$ and remove it from the set L

8: end for

9: for $n = 1, 2, \dots, N$ do

10: for $L_m \in L$ do

11: According to $t_{m,u} = \|L_m - \varphi_u\|$, calculate the distance between MD m and the centroid

12: Add L_m to the s th cluster $C_s = C_s \cup \{L_m\}$ according to $s = \text{argmin}(t_{m,u})$

13: Update φ_u according to (36)

14: end for

15: if $\text{SSE}(n) = \text{SSE}(n - 1)$ then

16: End loop

17: end if

18: end for

19: while $|C_u| > \delta$ do

20: Remove L_m with $\max(t_{m,u})$ from cluster C_u

21: Add the L_m with $\min(t_{m,w})$ to the neighboring C_w and $\forall C_w \neq C_u, C_w \in C$

22: end while

23: Output: Generated C_U clusters $\{C_1, C_2, \dots, C_U\}$

4.2 MADDPG Algorithm

All MDs of the system are divided into different clusters by an adaptive clustering algorithm for better further solving of the objective function. However, within clusters, both MD m and UAV u must make decisions at each time slot, with the objective function also involving both continuous and discrete variables. Consequently, minimizing the computational overhead for MD m during computation offloading within clusters is framed as an MINLP problem. Traditional methods for addressing such problems are often com-

plex and challenging, making it difficult to achieve optimal solutions. In contrast, multi-agent deep reinforcement learning (MADRL) has shown promise in effectively training models to identify optimal solutions. Hence, in this subsection, we first established the multi-UAV assisted computation offloading problem as an MDP and then utilized the MADDPG algorithm in conjunction with the system computation offloading model to find the optimal solution.

1) MDP Construction

We considered each UAV in the system as an agent and constructed a Markov decision process, which consists of the tuple $\{S, O, A, T, R\}$. Multiple agents in the system interact with the environment. Assuming that the state of the environment at the time slot t is $s(t) \in S$, each agent obtains its observation $o(t) \in O$ based on $s(t)$ and makes a corresponding action $a(t) \in A$, while the environment generates an immediate reward $r(t) \in R$ based on the agent's action as feedback to evaluate the merits of the current action and updates the next state with a probability $T[S(t+1)|S(t), A(t)]$. The new state depends only on the current state and the actions taken by each agent, which are executed based on the policy $\pi[a(t)|o(t)]$ with the goal of learning the optimal policy to maximize the long-term cumulative reward.

2) Observation Space

For each cluster, the observation range of UAV u is somewhat restrictive, and all of them can only observe part of the state information of the responsible cluster, while the global state information of the system and the actions of other UAVs are unknown to UAV u . Therefore, for the UAV u in the time slot t to observe the cluster information there is the location information of the UAV u , location information of the MDs within the cluster, information about the tasks generated by the MDs, number of CPU cycles required for MD, information about the sub-bandwidth resources allocated by the system to the cluster C_u , and channel state information of UAV u and MDs. Then the observed value of

the agent u can be written as

$$o_u(t) = \{q_u^{\text{UAV}}(t), q_m^{\text{MD}}(t), D_m(t), \Omega_m(t), b_u, g_{m,u}(t), \forall m \in M, t \in T\} \quad (37)$$

Note that the agent u values of the observations are changing at different time slots, which means that the MDs are moving randomly and generating new tasks, and the UAV needs to follow suit and adjust the flight attitude to more closely match the real situation.

3) Action Space

The UAV u will take corresponding actions based on the observed information, at the current time slot t first needs to determine whether the UAV u has established communication links with all MDs in the cluster, the UAV u needs to calculate how many proportions of the total tasks, UAV u allocates how much computational resources to each MD and the UAV u determines its own flight angle and flight speed according to the position of the MD. Then the action value of the agent u can be written as

$$a_u(t) = \{a_{m,u}(t), \zeta_{m,u}(t), \beta_{m,u}, V_{\text{UAV}}(t), \forall m \in M, t \in T\} \quad (38)$$

4) State Space

The state of the system can be viewed as the set of all UAVs' observations, which contains the positions of all UAVs, the positions of all MDs, their tasks, and other information, and then the state of the system is written as

$$s(t) = \{o_u(t) | u \in U\} \quad (39)$$

5) Reward Function

To fulfill the computation offloading performance requirements of MDs, the design of an appropriate reward function is crucial for the learning of the agent. In this context, the reward function $r_u(t)$ at the time slot t usually corresponds to the desired objective function of minimizing the average computational overhead between each MD and UAV within the cluster. However, given that reinforcement learning typically aims to maximize rewards, the value of the objective function must be inversely related to

the reward value. To make it more realistic, we introduced the decay coefficients $\kappa_{m,u}$ and $\kappa_{m,u} \in [0, 1]$. Therefore, the reward after the agent u which executes the action is written as

$$r_u(t) = -\kappa_{m,u} \frac{1}{TM} \sum_{m=1}^M \sum_{t=1}^T \delta_t T_m^{\text{total}}(t) + \delta_c E_{m,u}^{\text{total}}(t) \quad (40)$$

The reward function is used to evaluate the computation cost of UAV-based MD partial offloading decisions. When an MD opts for local processing, the reward is related to the delay and energy consumption incurred by computing resource usage; when a task is offloaded to a UAV, the reward is evaluated based on the offloading success rate, bandwidth utilization, as well as the delay and energy consumption resulting from transmission and UAV execution. A decay factor, $\kappa_{m,u}$, is introduced to adjust the reward difference between local processing and offloading tasks, thereby encouraging MDs to choose more efficient offloading strategies. The reward function comprehensively considers task completion time and energy consumption to optimize offloading decisions, effectively reducing the average computation cost for MDs.

4.3 MADDPG Algorithm Framework

In dynamically changing multi-agent environments, it is difficult for a single agent to adapt on its own, and collaboration and distributed learning are usually required to improve the adaptive ability of the overall system. Therefore, the algorithm of MADDPG, which is adopted as one of the schemes of MADRL, can be applied to complex multi-agent environments, where each of its agents can learn collaboratively and improve the network performance [29]. The MADDPG algorithm represents an adaptation and improvement of the single-agent DDPG algorithm for multi-agent settings, with the parameters of each network framework updated similarly to those in the DDPG algorithm. Both MADDPG and DDPG are based on the actor-critic framework, where each agent has its own actor and critic

networks, along with their respective target networks, as illustrated in Fig. 2. The actor network, responsible for policy formulation, is defined by parameter θ_u^Q . The critic network outputs estimate of the value function for the optimal state-action pair, which is denoted as $Q(s(t), a_1(t), a_2(t), \dots, a_U(t) | \varphi_u^Q)$, and is used to evaluate the performance of the actor network during the training phase. The parameter of the critic network is defined as φ_u^Q . In the MADDPG algorithm, the current agent trains two evaluation networks by cooperating with other agents to achieve collaborative decision making.

However, all agents share an experience replay buffer, and each agent can use the states and actions of the other agents to optimize its neural network so that each agent can fully consider the environment and the other agents to make a more reasonable decision. The MADDPG algorithm uses a centralized training and distributed execution model, the input of the critic network contains the observations and actions of all agents in a time slot, and the network parameters are updated in the centralized training mode but in the decentralized execution, the input of the actor network only contains its observations, which better improves the stability of the policy.

We will illustrate the steps of the MADDPG algorithm to solve the problem of minimizing the average computational overhead of MD, and the training process is similar for each agent. At each time step of each episode, the actor network of the agent u inputs the observation $o_u(t)$ into the actor network based on the current state $s(t)$ and the cluster environment, and the actor network will select the corresponding action $a_u(t)$ based on the policy, where the policy is consistent with the DDPG algorithm when selecting the action. However, to arrive at a better exploration, we constructed the action space by adding behavioral noise and then output the actions with the noise to the actor network. Thus, the output action is defined as

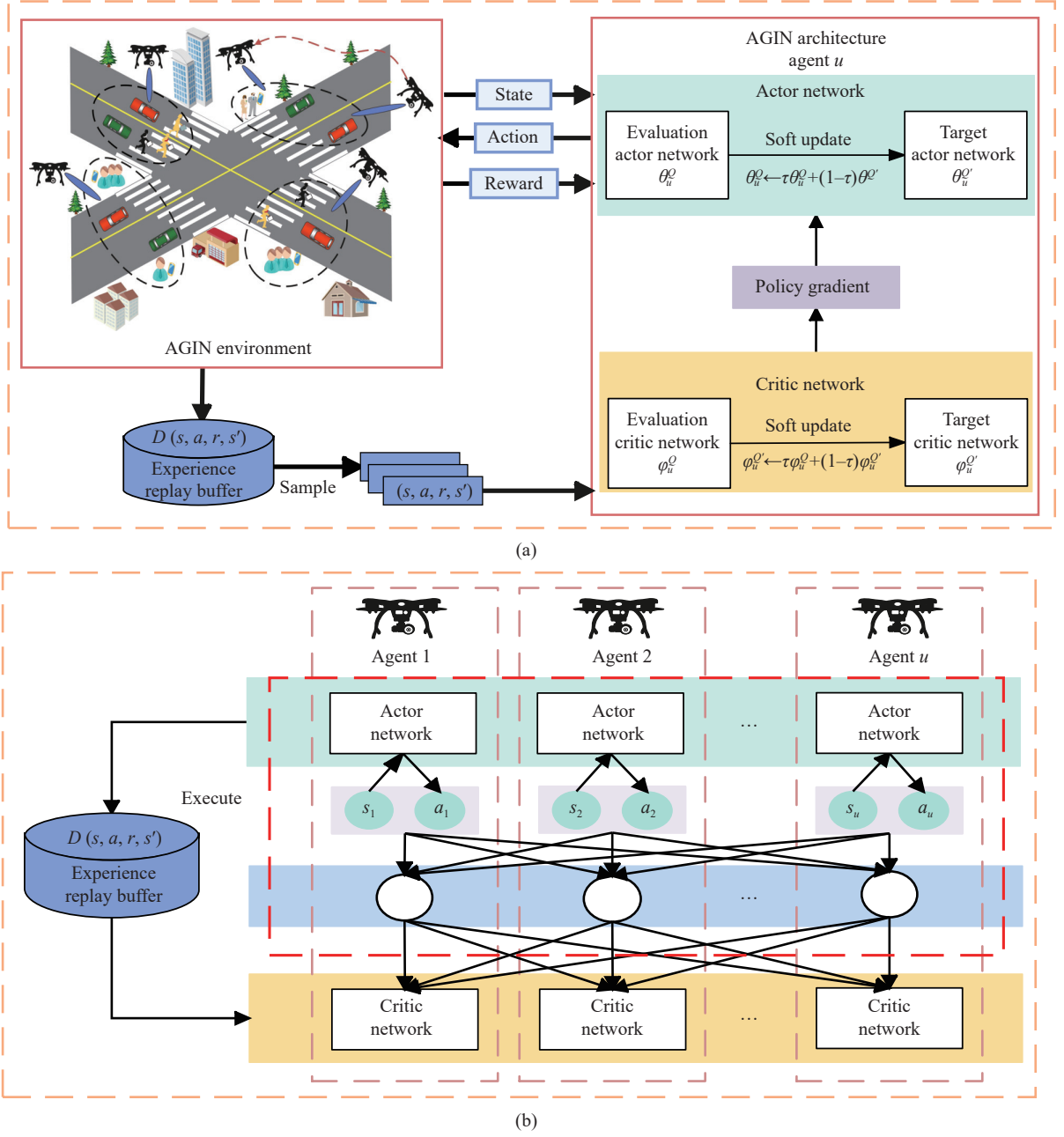


Fig. 2 Computation offloading and resource allocation algorithm architecture based on MADDPG: (a) distributed execution architecture; (b) centralized training framework

$$a_u(t) = \pi_u(s, o_u(t) | \theta_u^Q) + \epsilon_u \quad (41)$$

where ϵ_u is the noise $\epsilon_u \sim (0, \sigma_u^2)$ that obeys a normal distribution, $\pi_u(s(t), o_u(t) | \theta_u^Q)$ is then the action policy of the current time slot t which uses the agent u output from the current actor evaluation network. Following this process, we can obtain the action values $\{a_1(t), a_2(t), \dots, a_U(t)\}$ for all the agents in the system and give each agent feedback on the reward value $\{r_1(t), r_2(t), \dots, r_U(t)\}$, and enter the state $s(t+1)$ for the next

time slot in the given environment, and the agent u will in turn obtain the observation value $o_u(t+1)$ for the next time slot. The components of experience replay in the algorithm are $\langle o_u(t), \{a_1(t), a_2(t), \dots, a_U(t)\}, \{r_1(t), r_2(t), \dots, r_U(t)\}, o_u(t+1) \rangle$, which are put into the experience replay buffer D . When the accumulation of experience replay reaches a certain amount, the training of the neural network is started, and during each round of training, a small batch of samples

Z are taken from the experience pool and inputted into the agent's actor and critical network, and the network parameters are then updated by appropriate processing. We will also get a new record of Z strips, denoted as

$$\langle s_{t,i}, \{a_{1,i}, a_{2,i}, \dots, a_{U,i}\}, \{r_{1,i}, r_{2,i}, \dots, r_{U,i}\}, s_{t+1,i} | i = 1, 2, \dots, Z \rangle \quad (42)$$

and use this new record as an example. First, calculate the Temporal Difference (TD) error of the agent u

$$\varsigma_{u,i} = y_{u,i} - Q_u(s_{t,i}, a_{1,i}, a_{2,i}, \dots, a_{U,i} | \varphi_u^Q) \quad (43)$$

where $Q_u(*)$ is the Q value output from the critic evaluation network of agent u . We input the experience replay into the critic network of all agents and output the Q values of all agents, and then we got the target Q value of the agent u as

$$y_{u,i} = r_{u,i} + \gamma \left[Q'_u(s_{t+1,i}, a'_{1,i}, a'_{2,i}, \dots, a'_{U,i} | \varphi_u^{Q'}) \right]_{a_{j,i} = \pi_u(o_{j,t+1} | \theta_j^{Q'})} \quad (44)$$

where γ is the discount factor, $\gamma \in (0, 1]$. $Q'_u(*)$ is the Q value output from the critic target network of agent u ; $\pi(*)$ is the actor target network of agent u , which both have lagged updated network parameters to make training more stable. Therefore, the loss function for training Q value function is defined as

$$\text{loss}(\varphi_u^Q) = \frac{1}{Z} \sum_{i=1}^Z [y_{u,i} - Q_u(s_{t,i}, a_{1,i}, a_{2,i}, \dots, a_{U,i} | \varphi_u^Q)]^2 \quad (45)$$

Based on the above equation, the critic network for the agent u can be updated. The parameters of the critic network are adjusted through gradient descent

$$\varphi_u^Q \leftarrow \varphi_u^Q - \alpha^\varphi \nabla_{\varphi_u^Q} \text{loss}(\varphi_u^Q) \quad (46)$$

where α^φ is the learning rate of the critic network. Since the actor network is trained differently from the critic network. To make the critic network need to minimize the loss to approximate the value of the real Q , for the actor evaluation network, the gradient of the determined

policy gradient of the Q value needs to be used as a gradient rise to update the network parameters to maximize the value of the action, and then the action gradient of the agent u is

$$\nabla_{\theta_u^Q} J = \frac{1}{Z} \sum_{i=1}^Z [\nabla_{\theta_u^Q} \pi_u(s_{t,i}, o_{u,t,i}(t) | \theta_u^Q) Q_u(s_{t,i}, a_{1,i}, a_{2,i}, \dots, a_{U,i} | \varphi_u^Q)] \quad (47)$$

Update actor network parameters based on gradient ascent

$$\theta_u^Q \leftarrow \theta_u^Q + \alpha^\theta \nabla_{\theta_u^Q} J \quad (48)$$

where α^θ is the learning rate of the actor network. The agent u of the actor target network and the subsequent parameter updates of the critic target network are consistent with the DDPG algorithm. Therefore, as the parameters θ_u^Q and φ_u^Q of the evaluation network are continuously updated, and the parameters $\theta_u^{Q'}$ and $\varphi_u^{Q'}$ of the target network are updated using soft updating (where a positive constant $\tau \ll 1$ close to 0 is used)

$$\theta_u^{Q'} \leftarrow \tau \theta_u^Q + (1 - \tau) \theta_u^{Q'} \quad (49)$$

$$\varphi_u^{Q'} \leftarrow \tau \varphi_u^Q + (1 - \tau) \varphi_u^{Q'} \quad (50)$$

Therefore, Algorithm 2 summarizes the dynamic offloading and resource allocation of multi-UAV collaborative MEC based on MADDPG.

Algorithm 2 Task offloading and resource allocation algorithm based on MADDPG

Input: Current system parameter: $M, U, C_U, T, q_u^{\text{UAV}}, q_m^{\text{MD}}, V_{\text{UAV}}, P_{m,u}, P_u^{\text{fly}}, g_{m,u}, D_m, \Omega_m, \beta_{m,u}, f_m^{\text{MD}}, f_{m,u}^{\text{UAV}}, a_{m,u}, \zeta_{m,u}$

Output: Optimized UAV flight trajectories and MD average computational cost.

Initialize: Agent's actor evaluation network and critic evaluation network parameters: θ_u^Q and φ_u^Q ; agent's actor target network and critic target network parameters: $\theta_u^{Q'}$ and $\varphi_u^{Q'}$; observation space $o_u(t)$, state space $s(t)$, action space $a_u(t)$ and reward $r_u(t)$; empirical replay buffer $\mathcal{D}$, discount factor γ , and soft update rate τ .

1: for episode = 0 to E do

```

2:   Reset environment to obtain initial observations
3:   for  $t = 1$  to  $T$  do
4:     for  $u = 1$  to  $U$  do
5:       Input current state  $s(t)$  and observation  $o_u(t)$ 
for each agent
6:       Execute the action  $a_u(t) = \pi_u(s, o_u(t)|\theta_u^Q) + \epsilon_u$ 
7:       Obtain immediate reward  $r_u(t)$  and new obser-
vations  $o_u(t+1)$ 
8:     end for
9:     Store  $\langle o_u(t), a_u(t), r_u(t), o_u(t+1) \rangle$  in  $D$ 
10:    Obtain  $s(t+1) \leftarrow s(t)$ 
11:    if experience replay buffer is full do
12:      for  $u = 1$  to  $U$  do
13:        Randomly select  $Z$  samples from the experi-
ence pool to create a mini-batch
14:        Update critic network parameters according
to Eqs. (45) and (46)
15:        Update actor network parameters according
to Eqs. (47) and (48)
16:        Soft update target network parameters:
17:         $\varphi_u^{Q'} \leftarrow \tau \varphi_u^Q + (1 - \tau) \varphi_u^{Q'}$ 
18:         $\theta_u^{Q'} \leftarrow \tau \theta_u^Q + (1 - \tau) \theta_u^{Q'}$ 
19:      end for
20:    end if
21:  end for
22: end for

```

4.4 Computational Complexity Analysis

The time complexity of the K-Means-based adaptive clustering algorithm mainly consists of three parts: initialization, iteration, and cluster member checking. In the initialization phase, the cluster centers are selected and initial assignments are made, with a complexity of $O(UM)$, where U is the number of UAVs and M is the number of clusters. During each iteration, the distance between each UAV or user and all cluster centers is computed, and the cluster members are updated, with a complexity of $O(UM)$. Assuming the maximum number of iterations is N , the total time complexity is $O(NUM)$. Additionally, checking and correcting the cluster members requires some computation, with a complexity of

$O(UM)$. In terms of space, the algorithm needs to store the center and member information for each cluster, so the space complexity is $O(U + M)$. Therefore, the overall time complexity is $O(NUM)$ and the space complexity is $O(U + M)$.

The computational complexity of the partial computation offloading algorithm based on MADDPG for training each actor with a batch of experience is $O(SL + L^2 + LA)$, where S represents the dimension of the state space, L is the number of neurons in the hidden layer, and A is the dimension of the action space. Similarly, the time complexity of the critic network can be expressed as $O(S + A)L + L^2 + L$. Since the target actor and target critic networks have the same network structure as the actor and critic networks, the algorithm complexity for a single agent can be expressed as $O[2(SL + L^2 + LA) + 2(S + A)L + L^2 + L]$. Therefore, the overall time complexity of the algorithm is $O[2UZ(SL + L^2 + LA) + (S + A)L + L^2 + L]$, where NN is the number of agents.

5 Simulation Results and Analysis

In this study, we verified the effectiveness of the MADDPG algorithm in improving the system performance and reducing the average computational overhead of MD through simulation and proved its good convergence. We compared the performance of the MADDPG algorithm with the DQN and actor-critic (AC) algorithms and found that the MADDPG algorithm performs well with the same parameter settings. In addition, we analyzed the effects of intra-cluster NOMA and OMA techniques on the computation offloading of MD tasks, and the results showed that NOMA has significant advantages in improving the system performance.

In the experimental parameter settings of this paper, parameters are chosen based on practical application requirements and current technological developments. The number of ground

users M is set between 6 and 18 to simulate user environments of different scales and test the algorithm's performance with varying user loads. An increase in user numbers leads to more computation offloading demands, thereby affecting task scheduling and resource allocation. The number of UAVs U is selected as 3, reflecting the typical UAV deployment scale in practical applications. While increasing the number of UAVs can improve offloading efficiency, it may also increase the complexity of resource management, especially in coordinating computing and communication resources. The task generation amount per MD D_m ranges from 1 to 5 Mbit, which reflects the varying computational capabilities and task volumes of different devices. Changes in task volume significantly impact offloading strategies, energy efficiency, and latency. We will further explore the impact of these parameter variations on experimental results, particularly in terms of optimizing offloading decisions and resource allocation in different network load and bandwidth conditions.

5.1 Experiment Parameter Setting

In the simulation, UAVs and MDs are randomly distributed within a 500 m $\times$ 500 m service area, and at the initial moment, UAVs are deployed near each cluster boundary according to the clustering algorithm, with a fixed flight altitude of 100 m, to provide services to all the MDs within the service area that requires computation offloading services. That is, starting from the first user in the vicinity of the UAV and flying sequentially to the last MD, it completes the task requests of all MDs within the cluster.

We conducted our experiments in a Python 3.8 development environment using PyTorch deep learning framework as a simulation platform. The hardware platform utilized a 13th Gen Intel(R) Core (TM) i9-13900HX processor and an NVIDIA GeForce RTX 4060 Laptop GPU graphics card. The parameters for the experimental simulations are detailed in Tab. 2 and Tab. 3.

Tab. 2 Simulation parameters

Parameter	Value	Parameter	Value
M	6–18	U	3
ϑ	0.2	D_m	1–5 Mbit
$V_{\text{UAV}}^{\text{max}}$	8 m/s	$V_{\text{MD}}^{\text{max}}$	1 m/s
$f_{m,u}^{\text{UAV}}$	1–9 GHz	f_m^{MD}	0.6 GHz
B^{total}	2–10 MHz	f_c	2 GHz
$P_{m,u}$	0.4–2 W	$\sigma_{m,u}$	–100 dBm

Tab. 3 Hyperparameter settings for MADDPG

Definition	Hyperparameter
Maximum episode	1000
Learning rate of actor	0.0001
Learning rate of critic	0.001
Memory size	10000
Batch size	64
Reward discount factor $\kappa_{m,u}$	0.9
Soft update factor τ	0.005
Action noise ϵ_u	0.2

5.2 Experiment Analysis

Fig. 3 first contrasts the average reward convergence of the multi-agent algorithm MADDPG against the single-agent algorithms DQN and AC. As the number of training sets increases, the reward value for each algorithm progressively improves and eventually remains stable. The MADDPG algorithm gradually converges to the near-optimal policy, and the corresponding state-action function gradually stabilizes, which can better deal with complex interactions and collaborations among multiple agents. The MADDPG algorithm outperforms the DQN and AC algorithms in every aspect of the average rewards, the convergence speed, and the stability. This

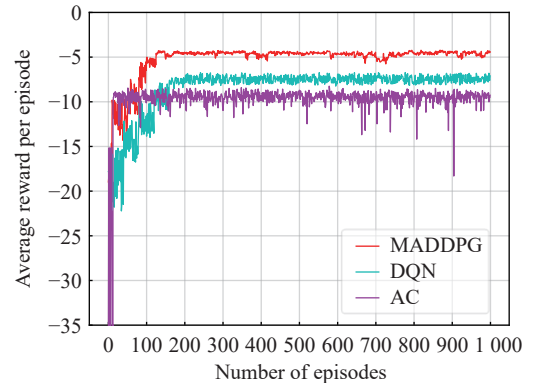


Fig. 3 Comparison of convergence of different algorithms

advantage is mainly because the MADDPG algorithm can fully utilize the characteristics of the NOMA-MEC network to better implement the hierarchical task partial offloading and network resource allocation strategies for the MDs within the clusters. In addition, the MADDPG algorithm solves the limitations of the DQN and AC algorithms in dealing with continuous behavioral space by introducing a policy network and a target network, which makes it possible to efficiently perform algorithmic updates without exhaustively enumerating the values of all behaviors in the continuous behavioral space. As a result, the MADDPG algorithm demonstrates greater adaptability and higher performance in complex environments, enabling it to perform well in multi-agent tasks.

Fig. 4 and Fig. 5 analyze the trends of the average delay and average energy consumption produced by each algorithm as the MD completes the task computation offloading. The results show that the MADDPG algorithm exhibits higher stability compared to the DQN and AC algorithms, and produces smaller values in both delay and energy consumption, thus effectively minimizing the average computational environments. Compared with the single-agent algorithm, the MADDPG algorithm can interact and collaborate effectively with all MDs in the cluster as well as other agents to rationally allocate resources. Specifically, each agent adjusts its flight attitude in real time according to the dynamic movement trend of the MDs in the cluster to satisfy the computation offloading requests of the MDs as much as possible. While the NOMA technique is employed to optimize the user's channel, increase the transmission rate, and reduce the interference inside and outside the cluster, the MADDPG algorithm enables the system to show better performance in terms of delay and energy consumption through more effective resource allocation and collaboration among the agents. This multi-agent collaboration significantly improves the efficiency and stability of the

system, enabling the MADDPG algorithm to minimize the computational overhead in a variety of complex environments.

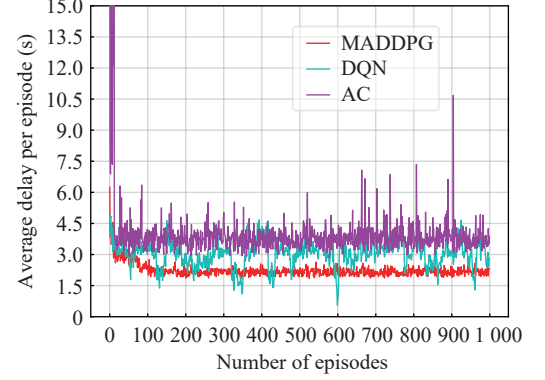


Fig. 4 Comparison of average delay of different algorithms

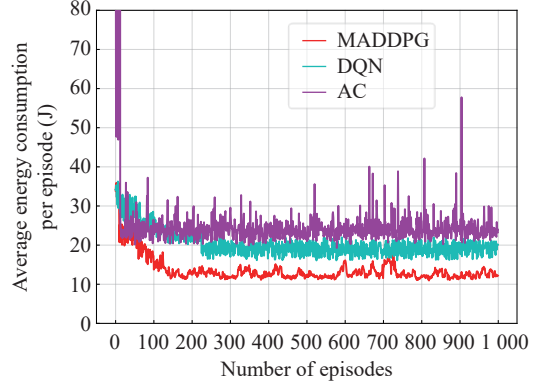


Fig. 5 Comparison of average energy consumption of different algorithms

Fig. 6 and Fig. 7 depict how the average computation delay and energy consumption of MDs change with the number of MDs in a cluster. The data reveals that as the number of MDs increases, both the average computation delay and energy consumption associated with task

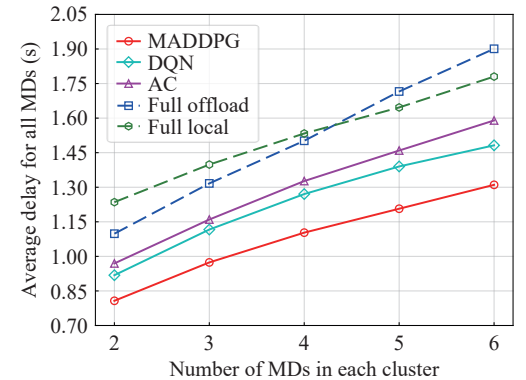


Fig. 6 Average delay against the number of MDs in a cluster

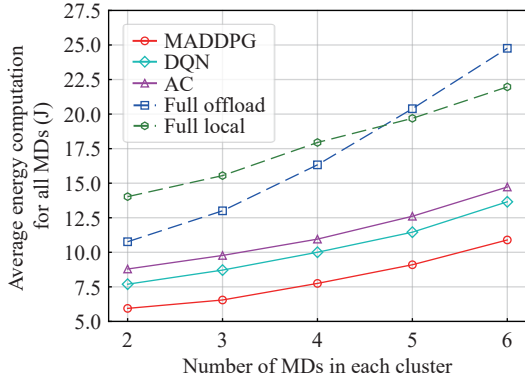


Fig. 7 Average energy consumption against the number of MDs in the cluster

offloading for each MD rise. The partial computation offloading algorithm based on MADDPG, as proposed in this paper, shows a marked improvement over the full offloading scheme, full local computation approach, and the DQN and AC algorithms in terms of reducing computation delay and energy consumption. This demonstrates that the proposed method is more effective in optimizing computation offloading and resource allocation for MDs.

Specifically, as the number of MDs in a cluster increases, the full computation offloading approach generates more delay and energy consumption than the partial computation offloading approach. This is because each UAV needs to provide computation offloading services for all MDs in the cluster it is responsible for, and the more MDs connected to the UAV, the less computational resources the UAV allocates, while the intra-cluster interference increases, putting more pressure on the MD's task transmission. The partial offloading scheme based on MADDPG combined with NOMA technology can effectively deal with these challenges. The MADDPG algorithm can offload part of the tasks to the UAV for computation according to the MD's computation offloading requirements, alleviating the computational pressure on the UAV. At the same time, NOMA technology is used to optimize the channel, reasonably allocate communication resources, and improve the utilization of spectrum resources. This not only reduces the computation delay and energy consumption but also

effectively copes with the intra-cluster interference and further reduces the average computation overhead of MD.

Fig. 8 analyzes the variation of the average data rate of each MD within a cluster against the uplink transmission power of the MDs. The results show that as each UAV is associated with MDs in an NOMA cluster, the data rate increases with the increase of uplink transmission power. In this paper, by analyzing the effect of three different schemes (MADDPG, MADDPG+OMA, and MADDPG+NOMA) on the rate based on the MADDPG algorithm, it is found that NOMA outperforms the other schemes in terms of increasing the MD transmission rate.

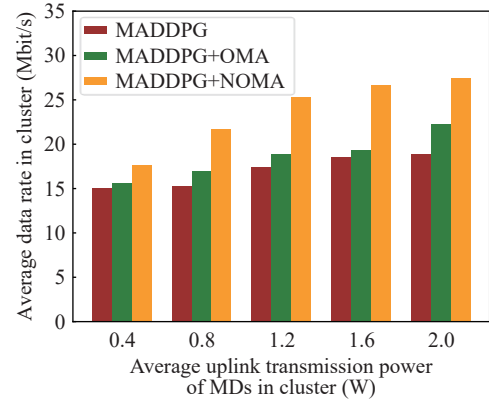


Fig. 8 Average data rate against transmission power of MDs

This is because NOMA allows multiple MDs to communicate on the same spectrum resource at the same time, distinguishing the devices by different power allocations, thus utilizing the spectrum resource efficiently. Compared with the traditional OMA technology, NOMA improves the spectrum efficiency and transmission rate by overlapping user signals and power allocation. This efficient resource utilization enables NOMA to significantly improve the MD transmission rate when facing high-density scenarios, and thus optimize the overall network performance. On the other hand, combining the MADDPG algorithm with NOMA technology not only optimizes the computational resource allocation through the policy network and the target net-

work but also improves the transmission rate and spectrum utilization using NOMA technology. This combination approach shows significant advantages when dealing with complex multi-agent environments and high-density scenarios.

Fig. 9 analyzes the trajectories of UAVs with assisted MDs for computation offloading. Three UAVs are deployed in the system. Due to the use of 3D graphics, it is difficult to completely display the flight trajectories of the three UAVs, but it is possible to observe the random distribution of MDs within each cluster at the same horizontal plane. Firstly, the clustering algorithm can reasonably divide the scope of the clusters according to the location and moving routes of the MDs, and each UAV is responsible for the computation of offloading requests for all the MDs within a cluster. The UAVs fly at a fixed altitude to adapt to the changing terrain of the urban environment and to achieve the line-of-sight conditions as much as possible, to efficiently plan the paths, to reduce the cost, and to ensure the safety of the flights. In addition, to avoid duplicated services, the UAV looks for and observes the MD's moving distance, direction, and other factors to dynamically adjust the flight path to fulfill all service requests from the first to the last MD in the cluster in real time. As a result, the flight trajectory of UAVs will show certain fluctuations when performing tasks. In this paper, through the combined use of the

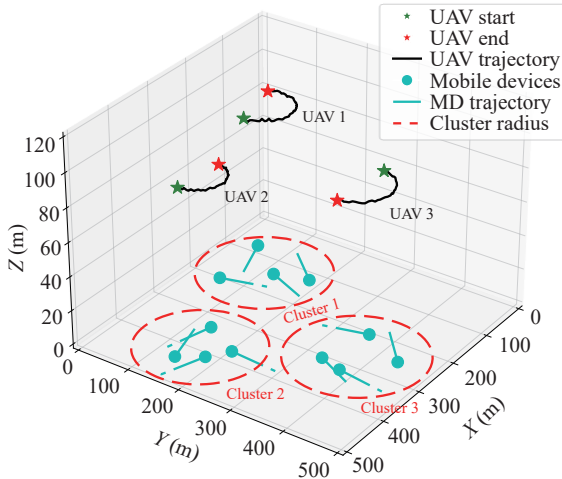


Fig. 9 UAV trajectory against MD in the multi-UAV situation

adaptive clustering algorithm based on MADDPG and K-Means, the UAV can flexibly adjust its flight strategy according to the state information of the MDs, thus effectively improving the efficiency of MD computation offloading.

Fig. 10 examines the average computation and communication cost of all MDs in relation to the total system bandwidth. As the system bandwidth continues to increase, the sub-bandwidth allocated to each cluster also increases. Consequently, the average cost associated with all schemes and algorithms, except for the full local computation scheme, decreases with more allocated sub-bandwidth. This is because a larger total bandwidth leads to a higher upload rate for tasks, which reduces transmission delay and energy consumption. As a result, increasing the total bandwidth diminishes the average delay and energy consumption of tasks, thereby lowering the computational overhead of MDs. In contrast, full local computation, which does not depend on data transmission, maintains relatively constant computational overhead. In a multi-agent environment, when MDs within a cluster have sufficient bandwidth resources, the UAV can effectively reduce competition and interference among MDs through optimal resource allocation, further reducing both average computation and communication costs. The MADDPG-based approach continues to outperform other schemes, demonstrating its effectiveness in adapting to variations in sub-bandwidth and significantly enhancing the efficiency of computation offloading for MDs.

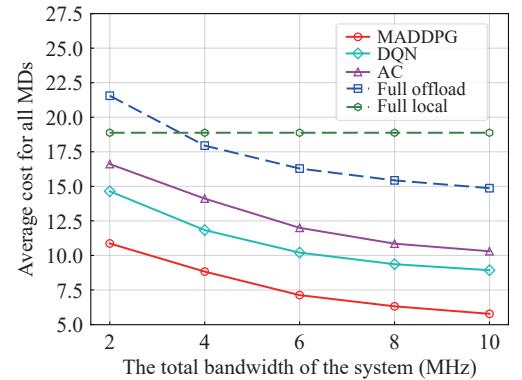


Fig. 10 Average cost against total system bandwidth

In addition, Fig. 11 shows that by analyzing the trend of MD's task offloading ratio versus UAV's computational capability, the results show that the average offloading ratio of MD in a cluster increases as the computational capability of UAV increases. This is because UAVs with high computational capability can process the tasks assigned to them more efficiently, which increases the task processing speed and thus reduces the computational overhead of the MD. Therefore, in this case, MDs are more willing to offload more tasks to UAV for execution to optimize the overall computational performance and resource utilization. On the contrary, if the UAV has low computational capability and processes MD data slowly, the overhead between local execution and offloading will increase. In this case, the MD may prefer local execution of tasks to avoid additional communication and processing delay.

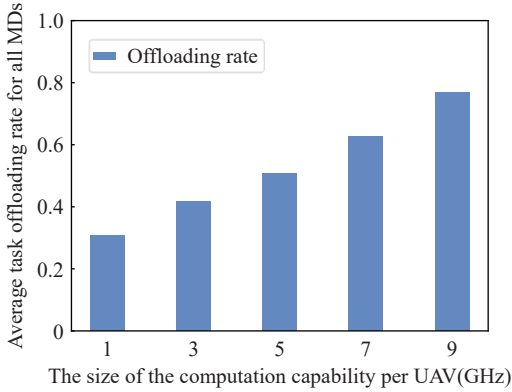


Fig. 11 Relationship between task offload rate and UAV computing capability

Fig. 12 and Fig. 13 analyze the trend of average delay and average energy consumption of MD with task bit size with different weights. The analysis reveals that both computational delay and energy consumption MDs increase with larger bit sizes. The delay weighting factor and energy consumption weighting factor are taken as $\delta_t = [0.2, 0.5, 0.8]$ and $\delta_e = [0.8, 0.5, 0.2]$ respectively. As shown in Fig. 12, when the delay weighting factor δ_t is increased, the average delay of MDs decreases, particularly as the bit size grows, and the advantage of delay perfor-

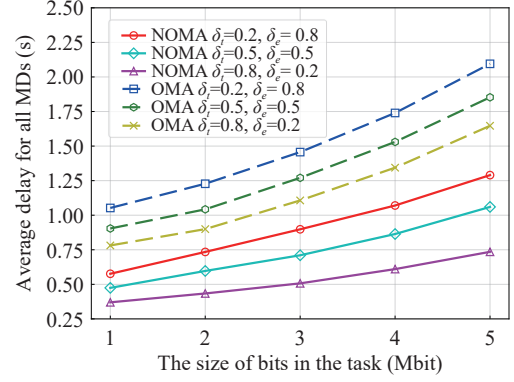


Fig. 12 Average delay against bit size of different weightings

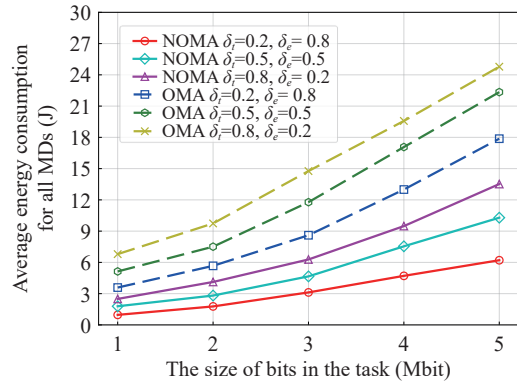


Fig. 13 Average energy consumption against bit size of different weightings

mance is more significant. This trend suggests that the UAV allocates more resources to reduce the delay of the MD. Similarly, as shown in Fig. 13, increasing the energy weighting factor δ_e results in reduced average energy consumption for MDs, with more pronounced energy performance as bit size increases, indicating that the UAV allocates more resources toward minimizing energy consumption. Furthermore, the NOMA scheme consistently outperforms the OMA scheme in reducing both delay and energy consumption, regardless of the variation of the weighting factors. Within a cluster, NOMA achieves lower delay and energy consumption through more efficient spectrum utilization. In contrast, the main drawback of OMA is the low efficiency of spectrum utilization, which makes it difficult to achieve the performance of NOMA in the same resource conditions because each user must occupy a separate band resource, which in turn leads to higher delay and energy consump-

tion. The NOMA scheme can achieve the overlapping use of the spectrum through resource sharing and user grouping, which can improve the overall performance.

6 Conclusion

This paper investigates the computation offloading problem in NOMA-based integrated air-ground MEC networks and proposes a multi-UAV-assisted computation offloading model to alleviate the load on ground base stations. By introducing NOMA technology, cluster interference is effectively eliminated when MDs offload tasks to UAVs, significantly enhancing MD transmission rates. To reduce the long-term average computational cost of MDs within clusters with limited computing and communication resources, we modeled the problem as an MINLP problem. To address this, we proposed a solution that combines an adaptive clustering algorithm based on K-Means with an MADDPG algorithm. First, the adaptive clustering algorithm clusters the MDs, and then the MADDPG algorithm jointly optimizes the UAV trajectories and MD task offloading decisions for each cluster. MADDPG optimizes decisions via multi-agent cooperation in a centralized training and distributed execution framework, effectively reducing delay and energy consumption, thereby lowering the computational cost for MDs. Comparative analysis with other single-agent algorithms and traditional offloading schemes shows that the proposed solution offers significant advantages in terms of average delay, energy consumption, and computational cost. Specifically, NOMA technology improves overall system performance by optimizing spectrum utilization and access efficiency, while MADDPG addresses the complexities of multi-agent cooperation and task offloading with its intelligent decision-making capabilities, ensuring system stability and efficiency.

References:

- [1] Z. Xiao, Y. Chen, H. Jiang, Z. Hu, J. C. Lui, G. Min, and S. Dustdar, "Resource management in UAV-assisted MEC: State-of-the-art and open challenges," *Wireless Networks*, vol. 28, no. 7, pp. 3305-3322, 2022.
- [2] M. W. P. Maduranga, V. Tilwari, R. M. M. R. Rathnayake, and C. Sandamini, "AI-enabled 6G Internet of things: Opportunities, key technologies, challenges, and future directions" *Telecom*, vol. 5, no. 3, pp. 804-822, 2024.
- [3] M. Li, J. Gao, L. Zhao, and X. Shen, "Adaptive computing scheduling for edge-assisted autonomous driving," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 6, pp. 5318-5331, 2021.
- [4] A. Mahmood, L. Beltramelli, S. F. Abedin, S. Zeb, N. I. Mowla, S. A. Hassan, and M. Gidlund, "Industrial IoT in 5G-and-beyond networks: Vision, architecture, and design trends," *IEEE Transactions on Industrial Informatics*, vol. 18, no. 6, pp. 4122-4137, 2021.
- [5] Y. Xu, T. Zhang, Y. Liu, D. Yang, L. Xiao, and M. Tao, "UAV-assisted MEC networks with aerial and ground cooperation," *IEEE Transactions on Wireless Communications*, vol. 20, no. 12, pp. 7712-7727, 2021.
- [6] C. Zhan, H. Hu, Z. Liu, Z. Wang, and S. Mao, "Multi-UAV-enabled mobile-edge computing for time-constrained IoT applications," *IEEE Internet of Things Journal*, vol. 8, no. 20, pp. 15553-15567, 2021.
- [7] C. Zhan, H. Hu, X. Sui, Z. Liu, and D. Niyato, "Completion time and energy optimization in the UAV-enabled mobile-edge computing system," *IEEE Internet of Things Journal*, vol. 7, no. 8, pp. 7808-7822, 2020.
- [8] M. Abd-Elnaby, G. G. Sedhom, E. S. M. El-Rabaie, and M. Elwekeil, "NOMA for 5G and beyond: Literature review and novel trends," *Wireless Networks*, vol. 29, no. 4, pp. 1629-1653, 2023.
- [9] X. Mu, Y. Liu, L. Guo, and J. Lin, "Non-orthogonal multiple access for air-to-ground communication," *IEEE Transactions on Communications*, vol. 68, no. 5, pp. 2934-2949, 2020.
- [10] M. Katwe, K. Singh, P. K. Sharma, C. P. Li, and Z. Ding, "Dynamic user clustering and optimal power allocation in UAV-assisted full-duplex hybrid NOMA system," *IEEE Transactions on Wireless Communications*, vol. 21, no. 4, pp. 2573-2590, 2021.
- [11] Y. Liu, S. Zhang, X. Mu, Z. Ding, R. Schober, N.

- Al-Dhahir, and X. Shen, "Evolution of NOMA toward next generation multiple access (NGMA) for 6G," *IEEE Journal on Selected Areas in Communications*, vol. 40, no. 4, pp. 1037-1071, 2022.
- [12] J. Wu, T. Xu, T. Zhou, X. Chen, N. Zhang, and H. Hu, "Feature-based spectrum sensing of NOMA system for cognitive IoT networks," *IEEE Internet of Things Journal*, vol. 10, no. 1, pp. 801-814, 2022.
- [13] C. Wu, X. Mu, Y. Liu, X. Gu, and X. Wang, "Resource allocation in STAR-RIS-aided networks: OMA and NOMA," *IEEE Transactions on Wireless Communications*, vol. 21, no. 9, pp. 7653-7667, 2022.
- [14] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and A. Nallanathan, "Deep reinforcement learning based dynamic trajectory control for UAV-assisted mobile edge computing," *IEEE Transactions on Mobile Computing*, vol. 21, no. 10, pp. 3536-3550, 2021.
- [15] S. Zarandi and H. Tabassum, "Delay minimization in sliced multi-cell mobile edge computing (MEC) systems," *IEEE Communications Letters*, vol. 25, no. 6, pp. 1964-1968, 2021.
- [16] X. Dai, Z. Xiao, H. Jiang, M. Alazab, J. C. Lui, S. Dustdar, and J. Liu, "Task co-offloading for D2D-assisted mobile edge computing in industrial internet of things," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 1, pp. 480-490, 2022.
- [17] J. Ren, K. M. Mahfujul, F. Lyu, S. Yue, and Y. Zhang, "Joint channel allocation and resource management for stochastic computation offloading in MEC," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 8, pp. 8900-8913, 2020.
- [18] M. A. Ali and A. Jamalipour, "UAV-aided cellular operation by user offloading," *IEEE Internet of Things Journal*, vol. 8, no. 12, pp. 9855-9864, 2020.
- [19] Q. Luan, H. Cui, L. Zhang, and Z. Lv, "A hierarchical hybrid subtask scheduling algorithm in UAV-assisted MEC emergency network," *IEEE Internet of Things Journal*, vol. 9, no. 14, pp. 12737-12753, 2021.
- [20] M. Wang, L. Zhang, P. Gao, X. Yang, K. Wang, and K. Yang, "Stackelberg-game-based intelligent offloading incentive mechanism for a multi-UAV-assisted mobile-edge computing system," *IEEE Internet of Things Journal*, vol. 10, no. 17, pp. 15679-15689, 2023.
- [21] Z. Hu, F. Zeng, Z. Xiao, B. Fu, H. Jiang, and H. Chen, "Computation efficiency maximization and QoE-provisioning in UAV-enabled MEC communication systems," *IEEE Transactions on Network Science and Engineering*, vol. 8, no. 2, pp. 1630-1645, 2021.
- [22] F. Pervez, A. Sultana, C. Yang, and L. Zhao, "Energy and latency efficient joint communication and computation optimization in a multi-UAV assisted MEC network," *IEEE Transactions on Wireless Communications*, vol. 23, no. 3, pp. 1728-1741, 2024.
- [23] W. Lu, Y. Ding, Y. Gao, Y. Chen, N. Zhao, Z. Ding, and A. Nallanathan, "Secure NOMA-based UAV-MEC network towards a flying eavesdropper," *IEEE Transactions on Communications*, vol. 70, no. 5, pp. 3364-3376, 2022.
- [24] I. Budhiraja, N. Kumar, S. Tyagi, and S. Tanwar, "Energy consumption minimization scheme for NOMA-based mobile edge computation networks underlaying UAV," *IEEE Systems Journal*, vol. 15, no. 4, pp. 5724-5733, 2021.
- [25] Y. Xu, T. Zhang, D. Yang, Y. Liu, and M. Tao, "Joint resource and trajectory optimization for security in UAV-assisted MEC systems," *IEEE Transactions on Communications*, vol. 69, no. 1, pp. 573-588, 2020.
- [26] N. T. Hoa, N. C. Luong, D. Van Le, and D. Niyato, "Deep reinforcement learning for multi-hop offloading in UAV-assisted edge computing," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 12, pp. 16917-16922, 2023.
- [27] K. K. Nguyen, T. Q. Duong, T. Do-Duy, H. Claussen, and L. Hanzo, "3D UAV trajectory and data collection optimisation via deep reinforcement learning," *IEEE Transactions on Communications*, vol. 70, no. 4, pp. 2358-2371, 2022.
- [28] B. Ma, Z. Liu, Q. Dang, W. Zhao, J. Wang, Y. Cheng, and Z. Yuan, "Deep reinforcement learning of UAV tracking control under wind disturbances environments," *IEEE Transactions on Instrumentation and Measurement*, vol. 72, no. 1, pp. 1-13, 2023.
- [29] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and L. Hanzo, "Multi-agent deep reinforcement learning-based trajectory planning for multi-UAV assisted mobile edge computing," *IEEE Transactions on Cognitive Communications and Networking*, vol. 7, no. 1, pp. 73-84, 2020.
- [30] S. Zhang, N. Yi, and Y. Ma, "A survey of computation offloading with task types," *IEEE Transac-*

tions on Intelligent Transportation Systems, vol. 25, no. 8, pp. 8313-8333, 2024.

- [31] Technical Specification Group Radio Access Network: Study on Enhanced LTE Support for Aerial Vehicles, 3GPP TR 36.777, 2017.
- [32] Y. Cui, D. Deng, C. Wang, and W. Wang, "Joint trajectory and power optimization for energy efficient UAV communication using deep reinforcement learning," in *IEEE INFOCOM 2021-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, Virtual Meeting, pp. 1-6, 2021.
- [33] Y. He, Y. Gan, H. Cui, and M. Guizani, "Fairness-based 3-D multi-UAV trajectory optimization in multi-UAV-assisted MEC system," *IEEE Internet of Things Journal*, vol. 10, no. 13, pp. 11383-11395, 2023.
- [34] J. Lyu, Y. Zeng, and R. Zhang, "UAV-aided off loading for cellular hotspot," *IEEE Transactions on Wireless Communications*, vol. 17, no. 6, pp. 3988-4001, 2018.



Jianbin Xue was born in 1973. He received B. S. degree in communication engineering from Sichuan University in 1997. He obtained the M. S. degree of signal and information system from Lanzhou University of Technology in 2005 and Ph. D. of control theory and control engineering from Lanzhou University of Technology in 2009. His main research direction is wireless communication theory and technology.



Peipei Mao received the B.S. degree in Lanzhou University of Technology in 2020, and is currently pursuing the M.S. degree in Lanzhou University of Technology, China, with a focus on mobile edge computing and nonorthogonal multiple access.



Luyao Wang received the B.S. degree in electronic information engineering from the College of Technical Engineering, Lanzhou University of Technology, China, in 2021, and is currently pursuing the M.S. degree in Lanzhou University of Technology, China, with a focus on wireless communication networks and edge computing for in-vehicle networks.



Qingda Yu received the B.S. degree in information countermeasures technology from Shenyang LiGong University in 2020, and is currently pursuing the M.S. degree in Lanzhou University of Technology, China, with a focus on mobile edge computing, and resource allocation for wireless communications.



Changwang Fan received the B.S. degree in communication engineering from Nanjing Institute of Technology in 2022, and is currently pursuing the M.S. degree in Lanzhou University of Technology, with a focus on intelligent reflective surfaces and mobile edge computing.